Package 'mypkg'

September 19, 2026

Type Package

Title What the Package Does (Title Case)

Version 0.1.0

Description More about what it does (maybe more than one line).
Continuation lines should be indented.

License What license is it under?

Encoding UTF-8

LazyData true

Imports stats,
quantreg,
conquer,
glmnet,
dplyr,
janitor,
magrittr,
Rcpp

LinkingTo Rcpp,
RcppArmadillo

RoxygenNote 7.3.3

Contents

lesreg 2
lesreg.fit 5
lesreg.highdim 7
lesreg.highdim.inference 9
lesreg.huber 11
lesreg.huber.fit 13
lesreg.ts 16
lesreg.wfit 17
print.lesreg.ts 19
print.summary.lesreg 19
print.summary.lesreg.huber 21
print.summary.lesreg.ts 22
summary.lesreg.huber 22

Index **24**

lesreg *Fitting Linear Expected Shortfall Regression Models*

Description

lesreg is used to fit linear expected shortfall regression models. It also computes the associated linear quantile regression fit used in the first step of estimation.

Usage

```
lesreg(formula, tau = 0.05, data, subset, weights, na.action,
       method = "br", model = TRUE, x = FALSE, y = FALSE,
       qr = TRUE, contrasts = NULL, eps = 1e-10,
       lm.tol = 1e-7, singular.ok = TRUE,
       upperTail = FALSE, ...)

## S3 method for class 'lesreg'
print(x, ...)
```

Arguments

formula	an object of class "formula": a symbolic description of the model to be fitted. The details of model specification are given under Details.
tau	the tail probability to be used. This is a single number strictly between 0 and 1. If upperTail = FALSE, tau is the lower-tail quantile level. If upperTail = TRUE, tau is the upper-tail probability and the corresponding quantile level is $1 - \tau$.
data	an optional data frame, list or environment containing the variables in the model. If not found in data, the variables are taken from environment(formula).
subset	an optional vector specifying a subset of observations to be used in the fitting process.
weights	an optional vector of observation weights to be used in the fitting process. The weights must be positive. If supplied, weighted quantile regression and weighted least squares are used in the two steps.
na.action	a function which indicates what should happen when the data contain NAs. The default is set by the na.action setting of options.
method	the method to be used for the first-step quantile regression. This is passed to rq.fit or rq.wfit. If method = "model.frame", the model frame is returned.
model, x, y, qr	logicals. If TRUE, the corresponding components of the fit, the model frame, the model matrix, the response, and the QR decomposition, are returned.
contrasts	an optional list. See the contrasts.arg argument of model.matrix.default.
eps	a non-negative numerical tolerance used when constructing the tail indicator.
lm.tol	tolerance passed to lm.fit or lm.wfit in the second-step least-squares fit.
singular.ok	logical. If FALSE, a singular fit in the second-step least-squares regression is an error.
upperTail	logical. If FALSE, the lower-tail expected shortfall is estimated. If TRUE, the upper-tail expected shortfall is estimated.
...	additional arguments to be passed to the low-level fitting functions, and ultimately to rq.fit or rq.wfit.

Details

Models for `lesreg` are specified symbolically. A typical model has the form `response ~ terms`, where `response` is the numeric response vector and `terms` is a series of terms which specifies a linear predictor for the conditional quantile and expected shortfall functions.

A formula has an implied intercept term. To remove this use either `y ~ x - 1` or `y ~ 0 + x`. See `formula` for more details of allowed formulae.

If `upperTail = FALSE`, the fitted model is

$$Q_\tau(Y \mid X) = X\beta,$$

together with

$$ES^L_\tau(Y \mid X) = E(Y \mid X, Y \leq Q_\tau(Y \mid X)) = X\theta.$$

If `upperTail = TRUE`, the fitted expected shortfall model is

$$ES^U_\tau(Y \mid X) = E(Y \mid X, Y \geq Q_{1-\tau}(Y \mid X)) = X\theta.$$

All of `weights`, `subset` and `na.action` are evaluated in the same way as variables in `formula`, that is first in `data` and then in the environment of `formula`.

`lesreg` calls the lower level functions `lesreg.fit` for unweighted fitting and `lesreg.wfit` for weighted fitting. For programming only, these functions may be called directly.

Value

`lesreg` returns an object of class `"lesreg"`.

The functions `summary.lesreg` and `print.lesreg` are used to obtain and print summaries of the results. The generic accessor functions `coef`, `fitted` and `residuals` can be used to extract useful features of the fitted object.

An object of class `"lesreg"` is a list containing at least the following components:

coefficients a named vector of expected shortfall regression coefficients.

beta.hat a named vector of coefficients from the first-step quantile regression.

theta.hat a named vector of coefficients from the second-step expected shortfall regression.

residuals the residuals from the second-step regression, that is the pseudo-response minus fitted expected shortfall values.

fitted.values the fitted expected shortfall values.

quantile.residuals the residuals from the first-step quantile regression.

quantile.fitted the fitted conditional quantile values.

es.fitted the fitted conditional expected shortfall values.

pseudo.response the pseudo-response used in the second-step least-squares regression.

pseudo.residuals the residuals from the second-step least-squares regression.

response.residuals the response minus fitted expected shortfall values.

indicator the tail indicator used to construct the pseudo-response.

rank the numeric rank of the fitted second-step linear model.

weights the specified weights, or `NULL` for unweighted fits.

df.residual the residual degrees of freedom.

call the matched call.

terms the terms object used.

contrasts only where relevant, the contrasts used.

xlevels only where relevant, a record of the levels of the factors used in fitting.

method the method used for the first-step quantile regression.

tau the tail probability used for estimation.

tail a character string indicating whether the lower or upper tail was fitted.

eps the numerical tolerance used in constructing the tail indicator.

lm.tol the tolerance used in the second-step least-squares fit.

singular.ok the value of singular.ok used in fitting.

qr if requested, the QR decomposition from the second-step least-squares fit.

y if requested, the response used.

x if requested, the model matrix used.

model if requested, the model frame used.

na.action where relevant, information returned by model.frame on the special handling of NAs.

quantile.fit the raw first-step quantile regression fit.

es.fit the raw second-step least-squares fit.

In addition, fitted objects contain x.fit and y.fit, the model matrix and response actually used by the low-level fitting routine.

Method

The function computes a two-step estimate. First, a linear quantile regression is fitted at level tau. Let $\hat{q}$ denote the fitted conditional quantile. The pseudo-response

$$w = \hat{q} + \tau^{-1} 1(y \leq \hat{q} + \epsilon)(y - \hat{q})$$

is then regressed on the same model matrix by least squares. The resulting coefficients estimate the linear expected shortfall function.

For upper-tail fitting, the lower-tail procedure is applied internally to the transformed response and model matrix. Fitted values are returned on the original response scale.

See Also

summary.lesreg, lesreg.fit, lesreg.wfit

rq, rq.fit and rq.wfit for quantile regression.

lm, lm.fit and lm.wfit for least-squares regression.

The generic functions coef, fitted and residuals.

Examples

```
set.seed(2026)
n <- 200
x1 <- rnorm(n)
x2 <- rnorm(n)
y <- 1 + 2 * x1 - x2 + rnorm(n)
dat <- data.frame(y = y, x1 = x1, x2 = x2)
fit <- lesreg(y ~ x1 + x2, data = dat, tau = 0.05)
fit
```

```
summary(fit)
fit.upper <- lesreg(y ~ x1 + x2, data = dat, tau = 0.05, upperTail = TRUE)
fit.upper
w <- stats::rexp(n)
fit.w <- lesreg(y ~ x1 + x2, data = dat, tau = 0.05, weights = w)
fit.w
mf <- lesreg(y ~ x1 + x2, data = dat, method = "model.frame")
head(mf)
```

`lesreg.fit` *Fitter Function for Linear Expected Shortfall Regression*

Description

This is the basic computing engine called by `lesreg` for unweighted linear expected shortfall regression. It works directly with a numeric design matrix and response vector.

For formula processing, missing-value handling, subsets, weights, and factor variables, use `lesreg`.

Usage

```
lesreg.fit(x, y, tau = 0.05, method = "br",
           upperTail = FALSE, eps = 1e-10,
           lm.tol = 1e-7, singular.ok = TRUE, ...)
```

Arguments

`x`	design matrix of dimension $n * p$.
`y`	vector of observations of length n, or a one-column matrix.
`tau`	the tail probability to be used. It must be a single number strictly between 0 and 1.
`method`	method of computation for the first-step quantile regression; passed to `rq.fit`.
`upperTail`	logical. If `FALSE`, the lower tail is fitted. If `TRUE`, the upper tail is fitted.
`eps`	non-negative numerical tolerance used in constructing the tail indicator.
`lm.tol`	tolerance passed to `lm.fit` in the second-step least-squares fit.
`singular.ok`	logical. If `FALSE`, a singular second-step fit is an error.
`...`	optional arguments passed to `rq.fit`.

Details

The function computes a two-step fit. First, a linear quantile regression is fitted by `rq.fit`. The fitted quantile is then used to form the pseudo-response

$$w = \hat{q} + \tau^{-1} 1(y \leq \hat{q} + \epsilon)(y - \hat{q}),$$

which is regressed on the same design matrix by `lm.fit`.

If `upperTail = TRUE`, the same lower-tail computation is applied to the transformed data $-y$ and $-x$. Fitted values are returned on the original response scale.

Fits without any columns return empty coefficient vectors and do not call the first-step or second-step fitting routines.

Value

a list with components

coefficients p vector of expected shortfall regression coefficients.

beta.hat p vector of first-step quantile regression coefficients.

theta.hat p vector of second-step expected shortfall regression coefficients.

x.fit design matrix used in fitting.

y.fit response used in fitting.

residuals n vector of second-step residuals.

fitted.values n vector of fitted expected shortfall values.

quantile.residuals n vector of first-step quantile residuals.

quantile.fitted n vector of fitted quantile values.

es.fitted n vector of fitted expected shortfall values.

pseudo.response n vector used as the response in the second-step least-squares fit.

pseudo.residuals n vector of residuals from the second-step fit.

response.residuals n vector of response residuals.

indicator n vector indicating the observations used in the tail.

rank integer, giving the rank of the second-step design matrix.

df.residual degrees of freedom of residuals.

assign the assignment attribute of the design matrix.

qr the QR decomposition from the second-step fit.

quantile.fit the object returned by the first-step `rq.fit` call.

es.fit the object returned by the second-step `lm.fit` call.

tail character string, either `"lower"` or `"upper"`.

contrasts the contrasts attribute of the design matrix, if any.

See Also

`lesreg`, `lesreg.wfit`, `rq.fit`, `lm.fit`

Examples

```
set.seed(2026)
n <- 200
x1 <- rnorm(n)
x2 <- rnorm(n)
y <- 1 + 2 * x1 - x2 + rnorm(n)
x <- model.matrix(~ x1 + x2)
fit <- lesreg.fit(x, y, tau = 0.05)
fit$theta.hat
fit.upper <- lesreg.fit(x, y, tau = 0.05, upperTail = TRUE)
fit.upper$theta.hat
```

`lesreg.highdim` *Fitting High-Dimensional Linear Expected Shortfall Regression Models*

Description

`lesreg.highdim` is used to fit high-dimensional linear expected shortfall regression models by a two-step penalized procedure. It also computes the associated penalized quantile regression fit used in the first step of estimation.

Usage

```
lesreg.highdim(x, y, alpha = 0.05, standardize = TRUE,
               tuning.method = c("CV", "BIC"), upperTail = FALSE)

## S3 method for class 'lesreg.highdim'
print(x, ...)

## S3 method for class 'summary.lesreg.highdim'
print(x, ...)

## S3 method for class 'lesreg.highdim'
summary(object, ...)
```

Arguments

`x`	numeric design matrix of dimension $n * p$, without an intercept column.
`y`	vector of observations of length n, or a one-column matrix.
`alpha`	the tail probability to be used. It must be a single number strictly between 0 and 1. This is the same tail level as `tau` in `lesreg`.
`standardize`	logical. If `TRUE`, the covariates are standardized before fitting the penalized model sequence. The coefficients are returned on the original scale by the underlying fitting functions.
`tuning.method`	character string specifying the method used to select the tuning parameters. Available choices are `"CV"` and `"BIC"`.
`upperTail`	logical. If `FALSE`, the lower-tail expected shortfall regression is fitted. If `TRUE`, the upper-tail expected shortfall regression is fitted by applying the lower-tail procedure to the negative response and then flipping the signs of the resulting estimators.
`...`	further arguments passed to or from other methods.
`object`	an object of class `"lesreg.highdim"`.

Details

This function implements a high-dimensional two-step estimator for linear expected shortfall regression. By default, the lower-tail expected shortfall regression model is fitted. The input matrix `x` should not contain an intercept column. Intercepts for the quantile and expected shortfall regressions are estimated separately.

In the first step, a penalized quantile regression is fitted at level `alpha`. If `tuning.method = "CV"`, the tuning parameter is selected by cross-validation using `conquer.cv.reg`. If `tuning.method = "BIC"`, the tuning parameter is selected by a BIC-type criterion using `conquer.reg`.

Let $\hat{q}$ denote the fitted conditional quantile. The pseudo-response

$$z = \hat{q} + \alpha^{-1} 1(y \leq \hat{q})(y - \hat{q})$$

is then regressed on the same design matrix by penalized least squares using `glmnet`. The resulting coefficients estimate the high-dimensional linear expected shortfall function.

If `upperTail = TRUE`, the upper-tail model is fitted by applying the same lower-tail procedure to `-y`. The fitted quantile and expected shortfall coefficients are then multiplied by `-1` and returned on the original response scale. Inference for a single coefficient can be obtained by `lesreg.highdim.inference`.

Value

`lesreg.highdim` returns an object of class `"lesreg.highdim"`.

An object of class `"lesreg.highdim"` is a list containing at least the following components:

theta0_hat intercept from the second-step expected shortfall regression fit.

theta_hat slope coefficients from the second-step expected shortfall regression fit.

beta0_hat intercept from the first-step quantile regression fit.

beta_hat slope coefficients from the first-step quantile regression fit.

resid residuals from the second-step fit, equal to the pseudo-response minus the fitted expected shortfall values.

lambda_beta selected tuning parameter from the first-step quantile regression fit.

lambda_theta selected tuning parameter from the second-step expected shortfall regression fit.

tau the tail probability used for estimation.

upperTail logical flag indicating whether the upper tail was fitted.

tail character string indicating whether the fitted model is for the `"lower"` or `"upper"` tail.

standardize logical flag indicating whether covariates were standardized in the penalized fitting routines.

tuning.method the tuning parameter selection method used.

call the matched call.

y the original response vector.

y.fit the response vector used internally for fitting. This is `y` for lower-tail fitting and `-y` for upper-tail fitting.

theta_hat_fit second-step slope coefficients on the internal fitting scale.

theta0_hat_fit second-step intercept on the internal fitting scale.

beta_hat_fit first-step slope coefficients on the internal fitting scale.

beta0_hat_fit first-step intercept on the internal fitting scale.

resid_fit second-step residuals on the internal fitting scale.

See Also

`lesreg`, `lesreg.highdim.inference`, `conquer.cv.reg`, `conquer.reg`, `glmnet`, `cv.glmnet`

Examples

```
set.seed(2026)
n <- 100
p <- 20
x <- matrix(rnorm(n * p), n, p)
y <- x[, 1] - x[, 2] + rnorm(n)

fit <- lesreg.highdim(x, y, alpha = 0.05)
fit

fit.bic <- lesreg.highdim(x, y, alpha = 0.05, tuning.method = "BIC")
fit.bic

fit.upper <- lesreg.highdim(x, y, alpha = 0.05, upperTail = TRUE)
fit.upper
```

---

lesreg.highdim.inference

Inference for High-Dimensional Linear Expected Shortfall Regression Models

---

Description

`lesreg.highdim.inference` performs inference on one coefficient of a high-dimensional linear expected shortfall regression model using a debiased estimator.

Usage

```
lesreg.highdim.inference(
  x,
  y,
  alpha,
  col = 1,
  res_est = NULL,
  conf.level = 0.95,
  standardize = TRUE,
  variance.method = c("RCV", "refit", "plug-in"),
  upperTail = FALSE
)

## S3 method for class 'lesreg.highdim.inference'
print(x, digits = max(5L, getOption("digits") - 2L), ...)
```

Arguments

`x`	numeric design matrix of dimension $n * p$, without an intercept column.
`y`	vector of observations of length n, or a one-column matrix.
`alpha`	the tail probability used for expected shortfall regression. It must be a single number strictly between 0 and 1. If `upperTail = FALSE`, `alpha` is the lower-tail probability. If `upperTail = TRUE`, `alpha` is the upper-tail probability.
`col`	integer specifying the index of the coefficient of interest.

res_est	optional fitted object returned by lesreg.highdim. If NULL, lesreg.highdim is called internally.
conf.level	confidence level for the reported confidence interval.
standardize	logical. If TRUE, covariates are standardized in the penalized fitting routines.
variance.method	character string specifying the method used to estimate the asymptotic variance. Available choices are "RCV", "refit" and "plug-in".
upperTail	logical. If FALSE, inference is performed for the lower-tail expected shortfall coefficient. If TRUE, inference is performed for the upper-tail coefficient by applying the lower-tail procedure to the negative response and flipping the sign of the debiased estimator and confidence interval.
digits	number of significant digits to use when printing.
...	further arguments passed to or from other methods.

Details

The fitted high-dimensional expected shortfall regression coefficients are penalized estimators and are not directly used for ordinary low-dimensional inference. For a specified coefficient indexed by col, this function computes a debiased estimator by projecting the covariate of interest on the remaining covariates and using the resulting residuals to form a decorrelated score.

If res_est is supplied, the first-step quantile regression and second-step expected shortfall regression estimates are taken from that object. Otherwise, lesreg.highdim is called internally.

Three variance estimation methods are available. The method "plug-in" uses a direct plug-in variance estimate. The method "refit" refits selected low-dimensional models on the selected supports. The method "RCV" uses a refitted cross-validation variance estimator.

Value

lesreg.highdim.inference returns an object of class "lesreg.highdim.inference".

An object of class "lesreg.highdim.inference" is a list containing

theta_debias the debiased estimator for the coefficient of interest.

Conf.int confidence interval for the debiased estimator.

col the index of the coefficient of interest.

tau the tail probability used for estimation.

conf.level the confidence level used for the reported confidence interval.

variance.method the variance estimation method used.

call the matched call.

upperTail logical flag indicating whether inference was performed for the upper tail.

tail character string indicating whether inference was performed for the "lower" or "upper" tail.

See Also

lesreg.highdim, lesreg, glmnet, cv.glmnet

Examples

```
set.seed(2026)
n <- 100
p <- 20
x <- matrix(rnorm(n * p), n, p)
y <- x[, 1] - x[, 2] + rnorm(n)

fit <- lesreg.highdim(x, y, alpha = 0.05)

inf <- lesreg.highdim.inference(
  x, y,
  alpha = 0.05,
  col = 1,
  res_est = fit,
  variance.method = "plug-in"
)
inf

fit.upper <- lesreg.highdim(x, y, alpha = 0.05, upperTail = TRUE)
inf.upper <- lesreg.highdim.inference(
  x, y,
  alpha = 0.05,
  col = 1,
  res_est = fit.upper,
  variance.method = "plug-in"
)
inf.upper
```

`lesreg.huber` *Fitting Adaptive Huber Expected Shortfall Regression Models*

Description

`lesreg.huber` fits linear expected shortfall regression models using quantile regression in the first step and adaptive Huber regression in the second step.

Usage

```
lesreg.huber(
  formula, tau = 0.05, data, subset, weights,
  na.action, method = "br",
  model = TRUE, x = FALSE, y = FALSE,
  contrasts = NULL, eps = 1e-10,
  huber.epsilon = 1e-4,
  huber.maxit = 500L,
  upperTail = FALSE, ...
)

## S3 method for class 'lesreg.huber'
print(x, digits = max(3L, getOption("digits") - 3L), ...)
```

Arguments

formula	an object of class "formula" specifying the response and linear predictor.
tau	tail probability. It must be a single number strictly between 0 and 1.
data	optional data frame, list, or environment containing the variables in the model.
subset	optional vector specifying observations to be used.
weights	observation weights. Weighted adaptive Huber expected shortfall regression is not currently implemented, and supplying this argument produces an error.
na.action	function specifying the handling of missing values.
method	method used for the first-step quantile regression; passed to rq.fit. If method = "model.frame", the model frame is returned.
model	logical. If TRUE, the model frame is retained.
x	logical. If TRUE, the model matrix is retained.
y	logical. If TRUE, the response is retained.
contrasts	optional list of contrasts passed to model.matrix.
eps	non-negative numerical tolerance used when constructing the tail indicator.
huber.epsilon	positive convergence tolerance used by the adaptive Huber regression algorithm.
huber.maxit	positive integer giving the maximum number of adaptive Huber iterations.
upperTail	logical. If FALSE, lower-tail expected shortfall is estimated. If TRUE, upper-tail expected shortfall is estimated.
...	additional arguments passed to the first-step quantile regression fitting routine.

Details

A linear quantile regression is first fitted at the specified tail probability. Let $\hat{q}_i$ be the resulting fitted conditional quantile. The pseudo-response

$$w_i = \hat{q}_i + \tau^{-1} 1(y_i \leq \hat{q}_i + \epsilon)(y_i - \hat{q}_i)$$

is then fitted using adaptive Huber regression.

Thus, lesreg.huber uses the same first-step quantile regression and pseudo-response construction as lesreg, but replaces the least-squares second step with adaptive Huber regression.

For upper-tail fitting, an equivalent lower-tail problem is fitted internally after a sign transformation.

The current implementation requires a model containing an intercept. Weighted adaptive Huber expected shortfall regression is not currently implemented.

Value

An object of class "lesreg.huber". Important components include:

coefficients adaptive Huber expected shortfall coefficients.

beta.hat first-step quantile regression coefficients.

theta.hat adaptive Huber expected shortfall coefficients.

fitted.values fitted expected shortfall values.

quantile.fitted fitted conditional quantile values.

pseudo.response pseudo-response used in the second step.

huber.tau adaptively selected Huber robustification parameter.

huber.iteration number of Huber iterations performed.

tau tail probability used in estimation.

tail either "lower" or "upper".

call matched function call.

See Also

lesreg.huber.fit, lesreg, lesreg.fit

Examples

```
set.seed(2026)
n <- 200
x1 <- rnorm(n)
x2 <- rnorm(n)

# Heavy-tailed errors
eps <- rt(n, df = 2.5)
y <- 1 + 2 * x1 - x2 + eps

dat <- data.frame(y = y, x1 = x1, x2 = x2)

fit <- lesreg.huber(
  y ~ x1 + x2,
  data = dat,
  tau = 0.1
)
fit

fit.upper <- lesreg.huber(
  y ~ x1 + x2,
  data = dat,
  tau = 0.1,
  upperTail = TRUE
)
fit.upper
```

---

lesreg.huber.fit *Fitter Function for Adaptive Huber Expected Shortfall Regression*

Description

This is the basic computing engine for unweighted linear expected shortfall regression with an adaptive Huber second-step estimator. It works directly with a numeric design matrix and response vector.

For formula processing, missing-value handling, subsets, and factor variables, use lesreg.huber.

Usage

```
lesreg.huber.fit(
  x, y, tau = 0.05, method = "br",
  upperTail = FALSE, eps = 1e-10,
  huber.epsilon = 1e-4, huber.maxit = 500L, ...
)
```

Arguments

x	numeric design matrix of dimension $n * p$. The current implementation requires exactly one intercept column.
y	numeric response vector of length n, or a one-column matrix.
tau	tail probability. It must be a single number strictly between 0 and 1.
method	method used for the first-step quantile regression; passed to rq.fit.
upperTail	logical. If FALSE, the lower-tail expected shortfall is fitted. If TRUE, the upper-tail expected shortfall is fitted.
eps	non-negative numerical tolerance used when constructing the tail indicator.
huber.epsilon	positive numerical convergence tolerance used by the adaptive Huber regression algorithm.
huber.maxit	positive integer giving the maximum number of adaptive Huber iterations.
...	additional arguments passed to rq.fit.

Details

The estimator is computed in two steps. First, linear quantile regression is fitted at level tau. Let $\hat{q}_i$ denote the fitted conditional quantile. The pseudo-response is

$$w_i = \hat{q}_i + \tau^{-1} 1(y_i \leq \hat{q}_i + \epsilon)(y_i - \hat{q}_i).$$

Unlike lesreg.fit, which fits the second step by least squares, lesreg.huber.fit regresses the pseudo-response on the covariates using adaptive Huber regression.

The adaptive Huber tuning parameter is estimated internally and is returned as huber.tau.

For upper-tail estimation, the corresponding lower-tail problem is fitted internally after a sign transformation, and fitted quantities are returned on the original response scale.

The current implementation requires an intercept because the underlying adaptive Huber fitting routine constructs its own intercept internally.

Value

A list containing:

coefficients adaptive Huber expected shortfall regression coefficients.

beta.hat first-step quantile regression coefficients.

theta.hat second-step adaptive Huber expected shortfall coefficients.

x.fit design matrix used in fitting.

y.fit response used internally in fitting.

residuals second-step pseudo-response residuals.

fitted.values fitted expected shortfall values.

quantile.residuals first-step quantile residuals.

quantile.fitted fitted conditional quantile values.

es.fitted fitted conditional expected shortfall values.

pseudo.response pseudo-response used in the adaptive Huber second step.

pseudo.residuals pseudo-response residuals.

response.residuals response minus fitted expected shortfall values.

indicator tail indicator used to construct the pseudo-response.

rank rank of the design matrix.

df.residual residual degrees of freedom based on the design matrix.

quantile.fit raw first-step quantile regression fit.

es.fit raw adaptive Huber fit returned by the internal C++ engine.

huber.tau adaptively selected Huber robustification parameter.

huber.iteration number of adaptive Huber iterations performed.

huber.epsilon Huber convergence tolerance used.

huber.maxit maximum number of Huber iterations allowed.

tail either `"lower"` or `"upper"`.

See Also

`lesreg.huber`, `lesreg.fit`, `lesreg`

Examples

```
set.seed(2026)
n <- 200
x1 <- rnorm(n)
x2 <- rnorm(n)
y <- 1 + 2 * x1 - x2 + rnorm(n)

x <- model.matrix(~ x1 + x2)

fit <- lesreg.huber.fit(
  x,
  y,
  tau = 0.1
)

fit$theta.hat
fit$huber.tau
```

`lesreg.ts` *Fit Autoregressive Quantile-Expected Shortfall Models*

Description

Fits a joint autoregressive quantile and expected shortfall model using the two-step estimation procedure and the weighted estimator.

Usage

```
lesreg.ts(
  y,
  tau = 0.05,
  qmodel = "SAV",
  emodel = qmodel,
  REP = 5,
  MAXITER = 1000,
  covMethod = 1,
  truncationMethod = 1
)
```

Arguments

`y`	A numeric time series.
`tau`	Quantile level. Must be strictly between 0 and 1.
`qmodel`	Autoregressive model for the conditional quantile. One of `"SAV"`, `"AS"`, `"GARCH"`, `"SAV12"`, or `"AS12"`.
`emodel`	Autoregressive model for the conditional expected shortfall. By default, the same model as `qmodel`.
`REP`	Number of repetitions used in the optimization procedure.
`MAXITER`	Maximum number of iterations for each optimization step.
`covMethod`	Method used for variance-covariance estimation.
`truncationMethod`	Method used for truncated conditional variance estimation.

Value

An object of class `"lesreg.ts"` containing the fitted quantile and expected shortfall models.

`lesreg.wfit` *Weighted Fitter Function for Linear Expected Shortfall Regression*

Description

This is the basic computing engine called by `lesreg` for weighted linear expected shortfall regression. It works directly with a numeric design matrix, response vector, and observation weights.

For formula processing, missing-value handling, subsets, and factor variables, use `lesreg`.

Usage

```
lesreg.wfit(x, y, weights = NULL, tau = 0.05, method = "br",
            upperTail = FALSE, eps = 1e-10,
            lm.tol = 1e-7, singular.ok = TRUE, ...)
```

Arguments

`x`	design matrix of dimension $n * p$.
`y`	vector of observations of length n, or a one-column matrix.
`weights`	vector of positive weights of length n to be used in the fitting process. If `NULL`, unit weights are used.
`tau`	the tail probability to be used. It must be a single number strictly between 0 and 1.
`method`	method of computation for the first-step weighted quantile regression; passed to `rq.wfit`.
`upperTail`	logical. If `FALSE`, the lower tail is fitted. If `TRUE`, the upper tail is fitted.
`eps`	non-negative numerical tolerance used in constructing the tail indicator.
`lm.tol`	tolerance passed to `lm.wfit` in the second-step weighted least-squares fit.
`singular.ok`	logical. If `FALSE`, a singular second-step fit is an error.
`...`	optional arguments passed to `rq.wfit`.

Details

The function computes a two-step weighted fit. First, a weighted linear quantile regression is fitted by `rq.wfit`. The fitted quantile is then used to form the pseudo-response

$$w = \hat{q} + \tau^{-1} 1(y \leq \hat{q} + \epsilon)(y - \hat{q}),$$

which is regressed on the same design matrix by `lm.wfit` using the same weights.

If `upperTail = TRUE`, the same lower-tail computation is applied to the transformed data $-y$ and $-x$. Fitted values are returned on the original response scale.

Fits without any columns return empty coefficient vectors and do not call the first-step or second-step fitting routines.

Value

a list with components

coefficients p vector of expected shortfall regression coefficients.

beta.hat p vector of first-step weighted quantile regression coefficients.

theta.hat p vector of second-step weighted expected shortfall regression coefficients.

x.fit design matrix used in fitting.

y.fit response used in fitting.

residuals n vector of second-step residuals.

fitted.values n vector of fitted expected shortfall values.

quantile.residuals n vector of first-step quantile residuals.

quantile.fitted n vector of fitted quantile values.

es.fitted n vector of fitted expected shortfall values.

pseudo.response n vector used as the response in the second-step weighted least-squares fit.

pseudo.residuals n vector of residuals from the second-step fit.

response.residuals n vector of response residuals.

indicator n vector indicating the observations used in the tail.

weights n vector of weights used in fitting.

rank integer, giving the rank of the second-step design matrix.

df.residual degrees of freedom of residuals.

assign the assignment attribute of the design matrix.

qr the QR decomposition from the second-step fit.

quantile.fit the object returned by the first-step `rq.wfit` call.

es.fit the object returned by the second-step `lm.wfit` call.

tail character string, either `"lower"` or `"upper"`.

contrasts the contrasts attribute of the design matrix, if any.

See Also

`lesreg`, `lesreg.fit`, `rq.wfit`, `lm.wfit`

Examples

```
set.seed(2026)
n <- 200
x1 <- rnorm(n)
x2 <- rnorm(n)
y <- 1 + 2 * x1 - x2 + rnorm(n)
x <- model.matrix(~ x1 + x2)
weights <- stats::rexp(n)
fit <- lesreg.wfit(x, y, weights = weights, tau = 0.05)
fit$theta.hat
fit.unit <- lesreg.wfit(x, y, tau = 0.05)
fit.unit$weights
fit.upper <- lesreg.wfit(x, y, weights = weights, tau = 0.05, upperTail = TRUE)
fit.upper$theta.hat
```

print.lesreg.ts *Print an Autoregressive Quantile-Expected Shortfall Fit*

Description

Print an Autoregressive Quantile-Expected Shortfall Fit

Usage

```
## S3 method for class 'lesreg.ts'
print(x, digits = max(5L, getOption("digits") - 2L), ...)
```

Arguments

x	An object of class "lesreg.ts".
digits	Number of significant digits used for printing.
...	Further arguments passed to or from other methods.

Value

Invisibly returns x.

print.summary.lesreg *Summarizing Linear Expected Shortfall Regression Fits*

Description

Summary method for class "lesreg".

Usage

```
## S3 method for class 'summary.lesreg'
print(x, digits = max(5L, getOption("digits") - 2L), ...)

## S3 method for class 'lesreg'
summary(object,
        method = c("default", "ind", "scl_N", "scl_sp", "rwb"),
        covariance = FALSE, level = 0.95, B = 500, ...)

## S3 method for class 'summary.lesreg'
print(x,
        digits = max(5L, getOption("digits") - 2L), ...)
```

Arguments

x	an object of class "summary.lesreg", usually a result of a call to summary.lesreg.
digits	the number of significant digits to use when printing.
...	further arguments passed to or from other methods.
object	an object of class "lesreg", usually a result of a call to lesreg.
method	character string specifying the method used to compute standard errors. There are currently five available methods: "default" the default sandwich estimator for the two-step expected shortfall regression fit. "ind" an estimator based on a conditional truncated variance approximation which is treated as independent of the covariates. "scl_N" an estimator based on a normal scale approximation to the conditional truncated variance. "scl_sp" an estimator based on a semiparametric scale approximation to the conditional truncated variance. "rwb" a random-weighting bootstrap estimator using exponential random weights.
covariance	logical flag indicating whether the estimated covariance matrix and, where available, its sandwich components should be returned.
level	confidence level for the reported confidence intervals.
B	number of random-weighting bootstrap replications. Used only when method = "rwb".

Details

The fitted "lesreg" object contains both the first-step quantile regression coefficients and the second-step expected shortfall regression coefficients. The summary method reports inference for the expected shortfall regression coefficients.

When method = "default", standard errors are computed from the sandwich form used for the two-step expected shortfall regression estimator. The methods "ind", "scl_N" and "scl_sp" use alternative estimates of the conditional truncated variance term. These estimates are computed internally using a conditional truncated variance estimator.

When method = "rwb", standard errors and confidence intervals are computed from random-weighting bootstrap replications. The argument B controls the number of bootstrap replications.

P-values are computed using the standard normal approximation.

Value

The function summary.lesreg computes and returns a list of summary statistics of the fitted expected shortfall regression model given in object, using the components call, terms, x.fit, y.fit, beta.hat and theta.hat from its argument, plus

coefficients a matrix with columns for the estimated coefficients, their standard errors, lower and upper confidence limits, and p-values.

residuals the residuals from the fitted second-step regression.

rdf the residual degrees of freedom.

tau the tail probability used for estimation.

method the variance estimation method used.

tail a character string indicating whether the lower or upper tail was fitted.

level the confidence level used for the reported confidence intervals.

cov the estimated covariance matrix, if `covariance = TRUE`.

Sigma_hat the estimated outer matrix in the sandwich covariance calculation, if `covariance = TRUE` and `method != "rwb"`.

Omega_hat the estimated middle matrix in the sandwich covariance calculation, if `covariance = TRUE` and `method != "rwb"`.

See Also

`lesreg`, `lesreg.fit`, `lesreg.wfit`,

Examples

```
set.seed(2026)
n <- 200
x1 <- rnorm(n)
x2 <- rnorm(n)
y <- 1 + 2 * x1 - x2 + rnorm(n)
dat <- data.frame(y = y, x1 = x1, x2 = x2)
fit <- lesreg(y ~ x1 + x2, data = dat, tau = 0.05)
summary(fit)
summary(fit, method = "ind")
summary(fit, method = "scl_N", covariance = TRUE)
summary(fit, method = "rwb", B = 100)
```

---

print.summary.lesreg.huber

Print Adaptive Huber Expected Shortfall Regression Summary

Description

Print method for objects of class `"summary.lesreg.huber"`.

Usage

```
## S3 method for class 'summary.lesreg.huber'
print(x, digits = max(5L, getOption("digits") - 2L), ...)
```

Arguments

`x`	an object of class `"summary.lesreg.huber"`, usually produced by `summary.lesreg.huber`.
`digits`	the number of significant digits to use when printing.
`...`	further arguments passed to or from other methods.

See Also

`summary.lesreg.huber`, `lesreg.huber`

print.summary.lesreg.ts *Summarizing Autoregressive Quantile-Expected Shortfall Fits*

Description

Summary method for objects of class "lesreg.ts".

Usage

```
## S3 method for class 'summary.lesreg.ts'
print(x, digits = max(5L, getOption("digits") - 2L), ...)

## S3 method for class 'lesreg.ts'
summary(object, level = 0.95, covariance = FALSE, ...)
```

Arguments

x	an object of class "summary.lesreg.ts".
digits	number of significant digits used for printing.
...	further arguments passed to or from other methods.
object	an object of class "lesreg.ts".
level	confidence level for the reported confidence intervals.
covariance	logical. If TRUE, the estimated covariance matrix is included in the returned object.

Value

An object of class "summary.lesreg.ts".

summary.lesreg.huber *Summarizing Adaptive Huber Expected Shortfall Regression Fits*

Description

Summary method for objects of class "lesreg.huber".

Usage

```
## S3 method for class 'lesreg.huber'
summary(
  object,
  alpha3 = 1,
  gamma = NULL,
  covariance = FALSE,
  level = 0.95,
  ...
)
```

Arguments

object	an object of class "lesreg.huber", usually a result of a call to lesreg.huber.
alpha3	positive numeric scalar used in the default calibration of the second robustification parameter gamma. The default is 1.
gamma	positive numeric scalar giving the truncation parameter used in robust covariance estimation. If NULL, it is set to (alpha3 * n / (p * log(n)))^(1/3).
covariance	logical flag indicating whether the estimated covariance matrix and its sandwich components should be returned.
level	confidence level for the reported confidence intervals.
...	further arguments passed to or from other methods.

Value

A list of class "summary.lesreg.huber" containing coefficient estimates, standard errors, confidence intervals, p-values, and the robust covariance tuning parameters.

Index

coef, *3, 4*
conquer.cv.reg, *8*
conquer.reg, *8*
cv.glmnet, *8*, *10*

fitted, *3, 4*
formula, *3*

glmnet, *8*, *10*

lesreg, 2, *5–8*, *10*, *12, 13*, *15*, *17, 18*, *20, 21*
lesreg.fit, *3, 4*, 5, *13–15*, *18*, *21*
lesreg.highdim, 7, *10*
lesreg.highdim.inference, *8*, 9
lesreg.huber, 11, *13*, *15*, *21*, *23*
lesreg.huber.fit, 13, *13*
lesreg.ts, 16
lesreg.wfit, *3, 4*, *6*, 17, *21*
lm, *4*
lm.fit, *2*, *4–6*
lm.wfit, *2*, *4*, *17, 18*

model.frame, *4*
model.matrix, *12*
model.matrix.default, *2*

options, *2*

print.lesreg, *3*
print.lesreg *(*lesreg*)*, 2
print.lesreg.highdim *(*lesreg.highdim*)*, 7
print.lesreg.highdim.inference *(*lesreg.highdim.inference*)*, 9
print.lesreg.huber *(*lesreg.huber*)*, 11
print.lesreg.ts, 19
print.summary.lesreg, 19
print.summary.lesreg.highdim *(*lesreg.highdim*)*, 7
print.summary.lesreg.huber, 21
print.summary.lesreg.ts, 22

residuals, *3, 4*
rq, *4*
rq.fit, *2*, *4–6*, *12*, *14*
rq.wfit, *2*, *4*, *17, 18*

summary.lesreg, *3, 4*
summary.lesreg *(*print.summary.lesreg*)*, 19
summary.lesreg.highdim *(*lesreg.highdim*)*, 7
summary.lesreg.huber, *21*, 22
summary.lesreg.ts *(*print.summary.lesreg.ts*)*, 22

terms, *4*